

CODEPIPELINE

Step 1 : Create buildspec.yml file

Before everything we have to create a [YAML](#) file in project's root directory. Name it as *buildspec.yml*.

We have to write all the commands necessary to build our app and deploy it, in this file. This is the YAML code for my simple React application.

Press enter or click to view image in full size

```
version: 0.1

phases:
  install:
    commands:
      - echo installing nodejs...
      - curl -sL https://deb.nodesource.com/setup_12.x | bash -
      - apt-get install -y nodejs
      - echo installing yarn...
      - curl -sS https://dl.yarnpkg.com/debian/pubkey.gpg | apt-key add -
      - echo "deb https://dl.yarnpkg.com/debian/ stable main" | tee /etc/apt/sources.list.d/yarn.list
      - apt install --no-install-recommends yarn
  pre_build:
    commands:
      - echo installing dependencies...
      - npm install
  build:
    commands:
      - echo building...
      - npm run build
artifacts:
  files:
    - "**/*"
  discard-paths: no
  base-directory: build
```

- In *phases* section there are three parts. In *install* part I have added commands to install [NodeJS](#) and [Yarn](#) which necessary to develop a React application (echo commands are just print commands).
- Then in *pre_build* part, install the dependencies by *npm install* command. Remember we installed Node previously. So, we can run npm commands in our environment.
- In *build* part, It will run the command *npm run build* to compile and build my React application. If you are a React developer you must know it will create a folder call "build" in root. We can host this build. (Similar to "dist" folder in Angular).
- Finally there's *artifacts* section. It defines the path to our "build" folder.

Simply artifacts are the things, what we pass from one stage to the next stage in CodePipeline. So here we are going to pass our build to Deployment stage which we are going to create later.

Step 2 : Push your code to GitHub

Now you should push your project to the GitHub. I'm not going to talk about [GIT](#) and how to create a repository here. Because this is about **AWS CodePipeline**. Yeah that's right, I should bold it

Step 3 : Create a S3 Bucket

We have to create an [AWS S3 Bucket](#). Simply S3 Bucket is a storage. We are going to put our React JS build here, so we can host it. Go to [S3](#) in AWS console and click on Create bucket.

Create bucket

Buckets are containers for data stored in S3. [Learn more](#)

General configuration

Bucket name

Bucket name must be unique and must not contain spaces or uppercase letters. [See rules for bucket naming](#)

Region

Bucket settings for Block Public Access

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

☐ Block **all** public access

Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

☐ Block public access to buckets and objects granted through **new** access control lists (ACLs)

S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.

☐ Block public access to buckets and objects granted through **any** access control lists (ACLs)

S3 will ignore all ACLs that grant public access to buckets and objects.

☐ Block public access to buckets and objects granted through **new** public bucket or access point policies

S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.

☐ Block public and cross-account access to buckets and objects through **any** public bucket or access point policies

S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.



Turning off block all public access might result in this bucket and the objects within becoming public
AWS recommends that you turn on block all public access, unless public access is required for specific and verified use cases such as static website hosting.

☒ I acknowledge that the current settings might result in this bucket and the objects within becoming public.

► Advanced settings

After creating the bucket you can upload files and folders to the bucket, and configure additional bucket settings.

Cancel

Create bucket

- Choose a Bucket name and a region. Make sure to *uncheck* “Block public access” and *check* the warning “I acknowledge” as shown above image. Now click *Create bucket*.
- After that, go to the *properties* tab of the Bucket. Click on *Static web hosting* then select *Use this bucket to host a website* option (On top of this box you can see the *URL Endpoint* for our application) and save.

Static website hosting

Endpoint : http://osusara-react-app.s3-website-us-east-1.amazonaws.com

☒ Use this bucket to host a website [Learn more](#)

Index document [i](#)

index.html

Error document [i](#)

error.html

Redirection rules (optional) [i](#)

☐ Redirect requests [Learn more](#)

☐ Disable website hosting

☒ Bucket hosting

Cancel Save

- Now go to the *Permissions* tab, then click on *Bucket Policy*. Here you have to paste bellow code snippet and save. This will make your S3 Bucket *public*. It's okay, unless we cant host it here.

```
{
  "Version": "2012-10-17",
  "Id": "Policy1555582784303",
  "Statement": [
```

```
{
  "Sid": "Stmt1555582783256",
  "Effect": "Allow",
  "Principal": "*",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::YOUR_S3_BUCKET_NAME/*"
}
```

That's it. Your storage is now all set.

Step 4 : Create AWS CodePipeline

Now go to the [CodePipeline](#) in AWS console and click on Create pipeline.

Press enter or click to view image in full size

Choose pipeline settings [Info](#)

Pipeline settings

Pipeline name
Enter the pipeline name. You cannot edit the pipeline name after it is created.

react-app

No more than 100 characters

Service role

☒ **New service role**
Create a service role in your account

☐ **Existing service role**
Choose an existing service role from your account

Role name

AWSCodePipelineServiceRole-us-east-1-react-app

Type your service role name

☒ Allow AWS CodePipeline to create a service role so it can be used with this new pipeline

► **Advanced settings**

Cancel **Next**

- Choose a Pipeline name.
- Keep the select *New service role*. It will create an IAM role to access the pipeline. In AWS we can assign [IAM roles](#) with necessary permissions for different service. Also, remember the *Role name* you put here.
- Don't mess with *Advanced settings* for now. Just click *Next*.

Step 5 : Create source provider

Now you'll get a screen similar to following image. Here we have to create a source provider. Simply defines from where our source code is coming.

Press enter or click to view image in full size

The screenshot shows the 'Add source stage' configuration interface. At the top, it says 'Add source stage' with an 'Info' link. Below this is a 'Source' section. Under 'Source provider', there's a dropdown menu set to 'GitHub' and a description: 'This is where you stored your input artifacts for your pipeline. Choose the provider and then provide the connection details.' Below the dropdown is a 'Connected' button. A green message box states: 'You have successfully configured the action with the provider.' Under 'Repository', there's a search bar with 'osusara/react-101' and a close button. Under 'Branch', there's a search bar with 'master' and a close button. Under 'Change detection options', there are two radio buttons: 'GitHub webhooks (recommended)' (selected) and 'AWS CodePipeline'. The 'GitHub webhooks' option has a description: 'Use webhooks in GitHub to automatically start my pipeline when a change occurs'. The 'AWS CodePipeline' option has a description: 'Use AWS CodePipeline to check periodically for changes'. At the bottom right, there are three buttons: 'Cancel', 'Previous', and 'Next'.

- Select GitHub as *Source provider*, because we are going to get our source code from GitHub repository.
- Then click on *Connect to GitHub* and insert your GitHub credentials to authorize.
- Now select your repository and the branch. You have to wait a few seconds to load your repositories, then you can select.
- Select GitHub webhooks as *Change detection option*. This will refresh our pipeline whenever we push changes to the repository. So our web application updates as soon as we make a change.

Step 6 : Create CodeBuild

- In here, select [AWS CodeBuild](#) as the build provider and select your region. Then click on *Create project*. This will open another browser window to create our project.

- In the new window give a *Project name*. *Description* and *tags* are not necessary.
- In *Environment* section, select each option as below image. Here we are creating the environment to build our application.

Environment

Environment image



Managed image

Use an image managed by AWS CodeBuild



Custom image

Specify a Docker image

Operating system

Ubuntu



The programming language runtimes are now included in the standard image of Ubuntu 18.04, which is recommended for new CodeBuild projects created in the console. See [Docker Images Provided by CodeBuild for details](#).

Runtime(s)

Standard

Image

aws/codebuild/standard:3.0

Image version

Always use the latest image for this runtime version

Environment type

Linux

Privileged

- ☐ Enable this flag if you want to build Docker images or want your builds to get elevated privileges

Service role



New service role

Create a service role in your account



Existing service role

Choose an existing service role from your account

Role name

codebuild-react-app-service-role

Type your service role name

► Additional configuration

Timeout, certificate, VPC, compute type, environment variables, file systems

Buildspec

Build specifications



Use a buildspec file

Store build commands in a YAML-formatted buildspec file



Insert build commands

Store build commands as build project configuration

Buildspec name - *optional*

By default, CodeBuild looks for a file named buildspec.yml in the source code root directory. If your buildspec file uses a different name or location, enter its path from the source root here (for example, buildspec-two.yml or configuration/buildspec.yml).

- Also, select *New service role* like before. It will create another IAM role for CodeBuild. Better to remember the role's name here as well. And no additional configurations.
- Select *Use a buildspec file* option in Buildspec section. If you remember we created the buildspec.yml file previously in project root directory. So we don't have to give the *buildspec name*. If we create a file with different name or in a sub directory, we have to give the name with the path.
- And keep select *CloudWatch logs* and click on *Continue to CodePipeline*. Then the new window will close.
- Now we are in our previous window again. Here click on *Next*.

Step 7 : Create deployment stage

- So this is the final stage. Here select *Amazon S3* as *Deploy provider* and select your region.
- Select the Bucket we previously created in 3rd step. Make sure to check *Extract file before deploy*. The reason is when the Build stage pass our React build to the Deployment stage as an artifact, it comes as a zip file.

Press enter or click to view image in full size

Add deploy stage [Info](#)

Deploy - optional

Deploy provider
Choose how you deploy to instances. Choose the provider, and then provide the configuration details for that provider.

Amazon S3 ▼

Region
US East (N. Virginia) ▼

Bucket
osusara-react-app ✕

Deployment path - optional

☒ Extract file before deploy
The deployed artifact will be unzipped before deployment.

► Additional configuration

Cancel Previous Skip deploy stage Next

- No any additional configuration. Just click on *Next*.

Step 8 : Review

This is not actually a step. Just shows us all the settings we selected previously in each stage. Make sure everything is okay and click on *Create pipeline*.

Now you can see your pipeline and it starts automatically when we create. Another thing, when the the Build stage is started, click on *Details* to see the terminal. You can see it runs the commands we wrote in buildspec.yml file. It's so satisfying to see everything is succeeded without any error

Press enter or click to view image in full size

The screenshot displays the AWS CodePipeline console interface. On the left, a sidebar menu shows the navigation structure: 'Developer Tools' with a close button, 'CodePipeline', and a list of resources including Source (CodeCommit), Artifacts (CodeArtifact), Build (CodeBuild), Deploy (CodeDeploy), Pipeline (CodePipeline), and Settings. The 'Pipeline' section is expanded, showing 'Getting started', 'Pipelines', 'Pipeline' (highlighted in orange), 'History', and 'Settings'. Below this is a search bar and a 'Feedback' link. The main content area is titled 'react-app' and shows a vertical flow of three stages: 'Source', 'Build', and 'Deploy'. Each stage is marked as 'Succeeded' with a green checkmark. The 'Source' stage uses 'CodeCommit' as the provider. The 'Build' stage uses 'AWS CodeBuild' and has a 'Details' link. The 'Deploy' stage uses 'Amazon S3'. Each stage card includes a 'Disable transition' button. The pipeline execution ID is visible for each stage, and the overall status is 'Succeeded'.

After all the stages succeeded, visit the Endpoint URL we saw in Step 3. Your web application should live now.